

DQ Compass

Implementation & Brief-Compliance Report

MBA-ESG / SG GSC Datathon 2026 — *DQ Compass: Building a Plug-and-Play Data Quality Control Layer for EUCs*

Working Group 1

Generated 2026-09-08

This document describes, in detail, every component of the DQ Compass prototype and cross-references it against the datathon brief (use case document V2 and its Appendices A, B and C). Section 5 is a line-by-line compliance matrix.

Layer	Component	Files
1 — Control Catalogue (DEFINES)	Rule repository + meta-catalogue validation	control_catalogue.csv, rule_authoring/schema.py
2 — Data Quality Engine (EXECUTES)	Dynamic control execution + threshold logic	dq_engine.py, controls.py, connectors.py
3 — Reporting Layer (REPORTS)	Scorecards, coverage, exceptions, supervisory mapping	reporting/scorecard.py, mapping.py
4 — Audit Layer (EVIDENCES)	Evidence Pack per run, hash-chained ledger, verification	evidence/<run_id>/, evidence/runs_ledger.jsonl
Governance / operations	Web app, audit changelogs, CLI, CI adapters	rule_authoring/, cli.py, docs/DEPLOYMENT.md

1 Executive summary

DQ Compass is a generic, reusable and auditable data quality control layer that plugs into any End-User Computing (EUC) tool without writing new code. It implements the four lifecycle stages the brief requires —

Definition -> Execution -> Reporting -> Audit — end to end, and adds the governance surface (a validated authoring application, hash-chained change logs, a run ledger, an independent-verification command and an auto-generated supervisory mapping) that turns a set of checks into a “simulation of industrial-grade Data Quality governance.”

What was built

- **Control Catalogue** — a 21-column CSV: the 14 mandatory Appendix A.2 attributes, a `regulatory_ref` column for the supervisory mapping, and six technical columns. `logic_definition`, `data_element` and `threshold` are generated.
- **Meta-catalogue** (`rule_authoring/schema.py`) — ~30 validation checks applied to every rule before it is saved: structure, controlled vocabularies, dataset declared, columns actually present (probed), and per-type parameter consistency; plus non-blocking warnings.
- **Data source registry** (`datasets_config.yaml` + `connectors.py`) — a pluggable connector abstraction. CSV and Excel are implemented; JSON, SQL database and SharePoint/Drive/URL are shown as selectors and saved with status preview.
- **Data Quality Engine** (`dq_engine.py` + `controls.py`) — reads the catalogue, dispatches by `control_type` to one of six generic control functions, applies the tolerance threshold, skips inactive rules, and never crashes on a missing source (it records an ERROR result and continues).
- **Audit Layer** — an Evidence Pack per run covering all ten Appendix B.2 components, an append-only `runs_ledger.jsonl` chained `previous_run_hash -> run_hash`, and `dqcompass verify <run_id>` which re-executes a past run from its stored config and dataset snapshots and asserts identical results.
- **Reporting Layer** — eight artefacts from `scorecard.py` (scorecard, exceptions, coverage, exception summary, dataset score, trend, alerts) and the Appendix C supervisory mapping from `mapping.py`.
- **Authoring web application** (`rule_authoring/`) — seven sections (Home, Catalogue, Data Sources, Runs, Mapping, Activity, Help) with full CRUD, inline validation, a coverage matrix, per-run drill-down, verify and sign-off, and two hash-chained audit changelogs.
- **CLI & deployment** (`cli.py`, `pyproject.toml`) — the `dqcompass` command (`run / verify / report / mapping / catalogue-validate / gate / serve`) plus cron, Airflow, CI-gate and notification recipes.
- **Tests** — 38 automated tests across three suites (schema 21, connectors 11, engine 6).

2 Architecture

2.1 The four layers and the files that implement them

Layer (brief §4)	Responsibility	Implementation
[1] Control Catalogue §4.2 / §6 / App. A	Structured repository of all DQ rules; parameterised, reusable, centrally governed	control_catalogue.csv (21 cols); authored & validated through rule_authoring/ (schema.py = the meta-catalogue); store.py = atomic writes + changelog
[2] Data Quality Engine §4.1 / §7.1	Execute multiple control types, apply rules dynamically to any dataset, produce structured reproducible outputs	dq_engine.py (dispatch, threshold, active filter, error handling); controls.py (6 generic functions); connectors.py (source loading)
[3] Reporting Layer §4.3 / §5.2 / App. C	Scorecards, exception reports, control coverage views; supervisory mapping	reporting/scorecard.py (8 outputs incl. scorecard.html); mapping.py (supervisory_mapping.csv); surfaced on the Runs and Mapping pages
[4] Audit Layer §4.4 / §7.2 / §7.3 / App. B	Reproducible, time-stamped, independently verifiable evidence of every execution	evidence/<run_id>/ (Evidence Pack); evidence/runs_ledger.jsonl (hash chain); dq_engine.verify_run(); signoff via the app

2.2 File map

File / directory	Purpose
control_catalogue.csv	The rule repository (21 canonical columns).
datasets_config.yaml	The data source registry: per source a connector type, status and config.
connectors.py	Connector abstraction + registry. CSV/Excel implemented; JSON/SQL/URL preview.
controls.py	Six generic control functions + CONTROL_REGISTRY (control_type -> function).
dq_engine.py	The engine: run(), per-rule execution, Evidence Pack, run ledger, verify_run().
reporting/scorecard.py	Reporting Layer: scorecard.csv/.html, exceptions_detail.csv, coverage.csv, exception_summary.csv, dataset_score.csv, trend.csv, alerts.json.
mapping.py	Appendix C supervisory mapping (supervisory_mapping.csv).
cli.py	The dqcompass console command.
rule_authoring/schema.py	The meta-catalogue: validation of every rule + generation of the auditor-readable columns + inline help text.
rule_authoring/store.py	Catalogue persistence: atomic CSV write, hash-chained catalogue_changelog.jsonl, CRUD, dataset re-targeting on rename.
rule_authoring/source_schema.py	Validation of a data source registration + plain-language summary.
rule_authoring/source_store.py	Data source persistence + data_sources_changelog.jsonl + the retire / reactivate / purge / rename policy with rule cascades.
rule_authoring/app.py	The Flask web application (all routes).
rule_authoring/templates/, static/	The 12 page templates + CSS/JS (design-token system, light/dark, list filters).
migrate_catalogue.py / migrate_sources.py	One-time schema migrations (also regenerate derived columns).
docs/ARCHITECTURE.md, docs/DEPLOYMENT.md	Design rationale + deployment recipes.
tests/	test_schema.py (21), test_connectors.py (11), test_engine.py (6).
sample_data/	orders.csv, orders_reference.csv, customers.csv with 6 deliberate anomalies.

2.3 Data flow of one run

- **Load** — DQEngine reads control_catalogue.csv and datasets_config.yaml.
- **Per active rule** — resolve the connector, load the dataset, compute its SHA-256 hash and write an exact copy to snapshots/<name>.csv; call the generic control function for the rule's control_type; apply threshold_pct.

- **Write per rule** — `<rule>_result.json` (status, metrics, config snapshot, resolved execution parameters, dataset hashes) and `<rule>_exceptions.csv`.
- **Write per run** — `system_trace.json`, `run_log.jsonl` + `run.log`, then `run_summary.json` with `run_hash = SHA-256(previous_run_hash + canonical(run_summary))`; append `runs_ledger.jsonl`.
- **Verify (on demand)** — `verify_run()` re-executes the stored rule configuration against the stored snapshots and asserts status + exception counts are unchanged; writes `verification.json`.

3 Component detail

3.1 Control Catalogue — control_catalogue.csv

One row per rule. Columns 1–14 are the mandatory Appendix A.2 attributes; regulatory_ref feeds the Appendix C mapping; the remaining six are technical. logic_definition (pseudo-SQL), data_element and threshold (plain text) are **generated** from the control type and parameters when a rule is saved, so the CSV is self-describing for an auditor without exposing the technical params JSON.

#	Column	Meaning
1	rule_id	Unique, stable identifier (e.g. DQ07). Key in catalogue, evidence and audit log.
2	control_name	Short readable name (3–80 chars).
3	control_type	One of the 6 DQ dimensions. Drives which parameters apply.
4	description	Business objective of the control (>=10 chars).
5	logic_definition	Formal rule logic as pseudo-SQL. <i>Generated</i> .
6	dataset_scope	Dataset the rule applies to (must be declared in datasets_config.yaml).
7	data_element	Field(s) tested. <i>Generated</i> from params.
8	threshold	Acceptance criteria in plain words. <i>Generated</i> .
9	severity	High / Medium / Low — drives prioritisation and the traffic light.
10	frequency	Execution periodicity (controlled vocabulary of 7 values).
11	owner	Responsible role (Data Owner / Data Steward / Control Owner ...).
12	output_type	Nature of the output on anomaly (error dataset / report / alert ...).
13	kpi	Quantifiable indicator (e.g. % completeness).
14	remediation_action	Action expected on failure (>=10 chars). Also carried into alerts.json.
15	regulatory_ref	Supervisory requirement satisfied (e.g. a BCBS 239 principle). Feeds the Appendix C mapping; default derived from the dimension when blank.
16	ref_dataset_scope	Reference dataset for Consistency / Reconciliation.
17	params	Machine parameters as JSON (field, keys, regex, thresholds, ...). Generated by the form; never hand-typed.
18	threshold_pct	Tolerance: % of records in anomaly below which the control still passes.
19	active	TRUE/FALSE. The engine skips FALSE; the rule stays in the catalogue and history.
20–21	created_at / updated_at	UTC timestamps.

The six DQ dimensions (control functions in controls.py)

Dimension	Function	Logic (pandas)	Params
Completeness	check_completeness	field is NA or blank after strip	field
Validity	check_validity	value not in allowed list / not matching regex / outside [min,max] (exactly one criterion)	field + one of {allowed_values, regex, min/max}
Uniqueness	check_uniqueness	df.duplicated(subset=keys) — the combination must be unique	keys (list — composite key)
Consistency	check_consistency	field value not present in the reference dataset's column (referential integrity)	field, ref_field, ref_dataset_scope
Timeliness	check_timeliness	date older than max_lag_days vs reference_date ('today' or a fixed ISO date)	field, max_lag_days, reference_date
Reconciliation	check_reconciliation	outer-join on key; break if a side is missing or src-ref /ref*100 > tolerance_pct	key, field, ref_field, ref_dataset_scope, tolerance_pct

Each function returns a standardised dict — {status: PASS|FAIL, metrics: {...}, exceptions: DataFrame}. No function knows a dataset or project name; it only receives column names, so one function serves any field of any dataset (brief §3 “Configurability”, App. A.4).

3.2 The meta-catalogue — validation applied before a rule is saved

`rule_authoring/schema.py` validates every rule. Blocking **errors** prevent the save; **warnings** are recorded and shown but allow the save. This is the brief's "rules must be explicit and testable" (App. A.4) made executable.

Blocking errors

Area	Checks (error code)
Rule ID	required; format letter + [A-Za-z0-9_-] (2–20); unique in the catalogue (duplicate)
Text fields	control_name / description / owner / kpi / remediation_action required and within length bounds (required, length)
Vocabularies	control_type, severity, frequency must be in the allowed sets (invalid)
Output type	required (a value outside the standard list is an <i>info</i> , not an error)
Dataset	dataset_scope required and declared in datasets_config.yaml (unknown_dataset)
Threshold	threshold_pct, if set, numeric and in [0, 100] (not_numeric, out_of_range)
Columns	field / keys / key / ref_field must exist in the target dataset — checked by reading a 50-row sample through the connector (unknown_column)
Completeness	field required (param_required)
Validity	field required; exactly one criterion among allowed_values / regex / range (validity_no_mode, validity_many_modes); regex must compile (bad_regex); min <= max (range_inverted)
Uniqueness	keys non-empty (param_required); no repeated column (duplicate_keys)
Consistency	field + ref_field required; ref_dataset_scope declared (param_required, unknown_dataset)
Timeliness	max_lag_days integer > 0 (non_positive, not_int); reference_date is 'today' or a valid ISO date (bad_date)
Reconciliation	key + field + ref_field required; ref_dataset_scope declared; tolerance_pct numeric in [0, 100] (not_numeric, out_of_range)

Warnings (rule saved, but flagged)

- High-severity control run at low frequency — weak risk coverage (critical_low_freq).
- Same (type, dataset, params, ref) as an existing rule — possible duplicate (duplicate_control).
- threshold_pct > 5 on a High-severity control (loose_critical).
- Reference dataset identical to the controlled dataset (self_reference).
- Overly permissive regex such as `. *` (permissive_regex); single allowed value (single_value); reconciliation key absent from the reference (key_absent_in_ref).
- Info: no threshold_pct set — the control is binary (no_threshold).

The same module also generates the auditor-readable columns (derive_logic_definition, derive_data_element, derive_threshold_text) and the plain-language explanation shown as a live preview in the form.

3.3 Data source registry & connectors

`datasets_config.yaml` holds, per named source, a connector type, a status (active / retired / preview) and a connector-specific config. This is the brief's "Separation of Concerns" (§3): the engine never knows *where* the data is until it is told.

Connector	Status	Config fields
CSV file	implemented	path, delimiter (incl. \t), header_row, encoding, quotechar, skip_rows
Excel file	implemented	path, sheet, header_row, skip_rows, nrows (uses openpyxl)
JSON file	preview	path, orient, lines, record_path
SQL database	preview	driver, host, port, database, schema, username, password, table / query
SharePoint / Drive / URL	preview	url, auth, token, format

- Each connector implements `load(config, root) -> DataFrame`, `probe(config, root) -> {ok, columns, row_count, sample, error}` and `validate(config) -> [issues]`.
- *Preview* connectors render the full field form (for the demo, incl. SQL credentials and the SharePoint link) but their `load()` raises and they are saved with `status: preview` — the engine skips them. Credentials on preview connectors are not persisted.
- The same `probe()` feeds the catalogue's column-existence check, so a rule can never reference a column its source does not expose.
- A new source type is one class in `connectors.py`; the engine, catalogue and reporting are untouched.

Delete policy (each step written to the hash-chained data source changelog):

Action	Effect on the source	Effect on dependent rules
Retire	status -> retired (reversible)	auto-deactivated; 'reactivate rules' brings them back
Purge	removed from the registry (typed PURGE confirmation)	definitions never deleted, only deactivated
Rename	key rewritten	dataset_scope / ref_dataset_scope rewritten on every dependent rule in one logged transaction

A rule pointing at a missing or retired source does **not** crash the engine run: it produces a `status: ERROR` result with a message and the run continues (brief §7.3).

3.4 Data Quality Engine — dq_engine.py

- **Dynamic dispatch** — for each row of the catalogue, look up `CONTROL_REGISTRY[control_type]` and call it with the parsed params; load the reference dataset too for Consistency / Reconciliation. No rule logic is hard-coded (brief §4.1).
- **Threshold** — `_apply_threshold()` turns a FAIL into a PASS when the exception rate $\leq$ `threshold_pct`, recording `exception_pct`, `threshold_pct` and `within_threshold` in the metrics. The same helper is used by verification so both compute status identically.
- **Inactive rules** — rows with `active=False` are skipped and logged as `rule_skipped`.
- **Resilience** — an unknown control type or an unavailable data source yields an ERROR result (with the reason) and the run continues.
- **Consistency across runs** — the engine is deterministic; two runs on the same inputs produce identical per-rule status, metrics and exception counts (verified by `tests/test_engine.py`).
- **Detailed metrics + severity** — every result carries the rule's `severity` and a metrics dict (`total_records`, `null_count`, `completeness_pct`, `duplicate_count`, `orphan_pct`, `break_count`, ...) — brief §4.1 “results must include severity levels and detailed metrics.”

3.5 Audit Layer — the Evidence Pack (Appendix B)

Each run writes evidence/`<run_id>`/. The table maps every mandatory Appendix B.2 component to the file that carries it.

Appendix B.2 component	Where it is in the Evidence Pack
Run ID	<code>run_summary.json</code> and every <code><rule>_result.json</code> (a UUID)
Execution Timestamp	<code>run_timestamp_utc / execution_timestamp_utc</code> (UTC ISO-8601)
Dataset Reference (version / snapshot / hash)	SHA-256 hash per dataset in each result plus an exact copy in <code>snapshots/<name>.csv</code>
Rule Configuration	<code>rule_configuration_snapshot</code> (<code>dataset_scope</code> , <code>ref_dataset_scope</code> , <code>params</code> , <code>threshold_pct</code>) in each result
Execution Parameters	<code>execution_parameters</code> — the <i>resolved</i> values, e.g. <code>reference_date_resolved: 2026-09-08</code>
Control Results (pass/fail, KPI values)	<code>status + metrics</code> in each result; aggregated in <code>run_summary.json</code>
Exception Dataset	<code><rule>_exceptions.csv</code> — the failing records, one file per failing rule
Logs (processing steps)	<code>run_log.jsonl</code> (structured) + <code>run.log</code> (plain text): <code>run_start</code> , <code>dataset_loaded</code> , <code>rule_start</code> , <code>rule_done / rule_error</code> , <code>system_trace_written</code> , <code>run_complete</code>
System Trace	<code>system_trace.json</code> : engine version, Python version, platform, host, pandas / pyyaml / openpyxl versions, SHA-256 of the catalogue, the config and <code>controls.py</code> , and the control function used per rule
Sign-off Fields	<code>signoff.json</code> — <code>reviewed_by</code> , <code>reviewed_at</code> , <code>decision</code> , <code>comment</code> (recorded from the Runs page)

Reproducibility, tamper-evidence and independent verification

- **Run ledger** — `evidence/runs_ledger.jsonl` appends one entry per run with `previous_run_hash` and `run_hash = SHA-256(previous_run_hash + canonical_json(run_summary))`. Altering any stored `run_summary.json` breaks the chain from that point on (brief §7.1 “results stored and versioned”, App. B.3).
- **Reconstruction** — every past run is fully contained in its folder: config snapshot + dataset snapshots + logs + trace. An independent party needs nothing else (§7.3, App. B.4).
- **Verification** — `dqcompass verify <run_id>` (or the ‘Verify this run’ button) reloads the stored rule configuration, re-executes each control against the stored snapshots/, applies the same threshold logic, and asserts `status + exception_count` are unchanged. It writes `verification.json` (`all_match` + a per-rule check table) and exits non-zero on any mismatch (§7.3 “independent verification of results”).
- **Retention note** — `system_trace.json` states that the prototype writes local files and that production would point evidence/ at versioned WORM storage (S3 Object Lock / immutable data-lake prefix).

3.6 Reporting Layer

`reporting/scorecard.py` reads the latest run and writes eight artefacts; `mapping.py` writes the Appendix C mapping. All are also surfaced inside the web app (Runs and Mapping pages).

Artefact	Content	Brief
scorecard.csv	One row per rule: status, traffic light (GREEN/ORANGE/RED/GREY for ERROR), severity, owner, exception count, all metrics	§4.3
scorecard.html	Standalone traffic-light scorecard for a demo without a BI tool	§4.3 / 5.2
exceptions_detail.csv	Every failing record from every rule, tagged with rule_id + severity — record-level drill-down	§4.3
coverage.csv	datasets x dimensions: which rules cover which (dataset, dimension) — the control coverage view	§4.3
exception_summary.csv	Exceptions grouped by severity / owner / control_type / dataset (long format, BI-friendly)	§4.3
dataset_score.csv	Severity-weighted composite DQ score per dataset (0–100): penalty = weight x min(1, exc_pct/100) for FAIL, weight for ERROR; weights High=5, Medium=2, Low=1	§4.3 / 8.2
trend.csv	Pass / fail / error per run over time, from the ledger	§4.3
alerts.json	One entry per failing High-severity control: rule, dataset, owner, exception count and the remediation action from the catalogue — failures captured, prioritised and <i>actionable</i>	§1.2
supervisory_mapping.csv	Appendix C.2/C.3: Requirement -> Control -> Evidence -> Output, plus the last run id + hash per rule	App. C

Supervisory mapping (Appendix C). `mapping.py` builds one row per active rule. The requirement is the rule's `regulatory_ref` if set, otherwise a default derived from the DQ dimension (phrased in BCBS 239 terms). The evidence column names the Evidence Pack files produced; the last-run columns link the control to its most recent execution (`run_id` + `run_hash`) — the Appendix C.3 control-to-evidence traceability. The Mapping page also lists the four Appendix C.4 principles (Traceability, Repeatability, Auditability, Governance) with how each is demonstrated.

3.7 Authoring web application — rule_authoring/

A Flask application with a sidebar shell and seven sections. All text is English; a light/dark theme toggle is persisted per browser.

Section	What it provides
Home	Dashboard: rule/source health tiles, the latest run as a pass/fail/error donut, a 'needs attention' list (validation errors, probe failures, retired sources with live dependents), the coverage heatmap, recent activity, a 'Run engine now' button.
Catalogue	The rules as filterable list items (search + status chips + control-type chips, each with a live count); every row expands to its logic, validation issues, owner / KPI / remediation. Full CRUD + activate/deactivate. Coverage matrix below.
Data Sources	One card per source: connector, status, plain-language summary, 'used by' rule chips; edit / retire / reactivate / rename / purge. The 'New data source' form has a connector-tile picker, progressive fields (advanced options collapsed) and a 'Test connection' preview.
Runs	Run history from the ledger (verified / signed-off badges) and a per-run drill-down: the Evidence Pack (hashes + chain), a result bar, per-rule cards (metrics, config snapshot, execution parameters, exception table), a Reporting card, a Verification table, the sign-off form, the system trace and the processing-log tail. Buttons: Verify this run, Record sign-off.
Reporting	Cross-run analytics with an audience filter (All / Control owners & risk / Data engineers). Business: composite DQ score + delta, a score-trend chart over the last runs, dataset health ranking, a 3-state coverage matrix (controlled+passing / controlled+failing / not controlled), open High-severity issues with remediation, exceptions by owner. Engineering: exceptions by dimension and by rule, a rule-reliability table (fail rate over the run window), a 'what changed since the previous run' diff, and the ERROR (broken source/config) list. All charts are dependency-free inline SVG. Power-BI-ready CSV/JSON downloads.
Mapping	The Appendix C.2/C.3 table + the C.4 principles + a supervisory_mapping.csv download.
Activity	The two audit changelogs merged into one timeline (filter Rules / Data sources), with per-entry field diffs and the file hash.
Help (/docs)	Field reference generated from the schema: the attributes, the 6 control types and their parameters, the meaning of each output_type / severity / frequency, the threshold, the connectors, the validation rules, the audit log.

Governance surface

- **Two hash-chained changelogs** — catalogue_change_log.jsonl and data_sources_change_log.jsonl. Every create / update / delete / (de)activate / retire / purge / rename appends an entry with the editor, the UTC timestamp, the before/after snapshot and the **SHA-256 of the file before and after**. The catalogue state at any past date is reconstructible (App. C.4 "Governance", brief §8.3).
- **Atomic writes** — the CSV and YAML are written to a temp file then `os.replace()`, so a killed process never leaves a half-written catalogue.
- **Deactivate, don't delete** — a retired rule keeps its definition and its history; it is simply out of the engine's scope.

3.8 CLI, packaging & deployment

Command	Purpose
dqcompass run	Execute the catalogue, write the Evidence Pack, append the ledger.
dqcompass verify <run_id>	Re-execute a past run from its snapshots; exit 1 on mismatch.
dqcompass report	(Re)generate the eight Reporting Layer files.
dqcompass mapping	Write reporting/supervisory_mapping.csv (Appendix C).
dqcompass catalogue-validate	Validate every rule against the meta-catalogue; exit 1 on error.
dqcompass gate --severity High	Run + exit 1 if any control at/above that severity broke — a CI quality gate.
dqcompass serve	Start the authoring web app (127.0.0.1:5001).

- `pyproject.toml` makes the project installable (`pip install -e .`) and puts `dqcompass` on the `PATH`.
- `docs/DEPLOYMENT.md` gives ready recipes: add the layer to an EUC in three lines; cron and Airflow schedules; a GitHub Actions workflow (`catalogue-validate` then `gate`); a notification loop that posts `alerts.json` to a channel with the remediation action.
- This is the brief's §8.4 “ease of deployment across EUCs / potential for industrialisation” and §9 “embed control logic into the data lifecycle.”

3.9 Tests

Suite	N	Covers
<code>tests/test_schema.py</code>	21	Every meta-catalogue check: valid rule per dimension, unknown dataset / column, Validity mode rules, bad regex, unknown keys, missing reference dataset, bad lag, out-of-range tolerance, duplicate / malformed id, cross-field warnings, the derivations.
<code>tests/test_connectors.py</code>	11	CSV default + custom delimiter + header offset + probe; Excel round-trip + sheet-not-found; preview connectors raise; SQL / URL shape validation; <code>normalize_entry</code> (flat vs rich); <code>load_source</code> missing / retired; <code>source_schema</code> validation.
<code>tests/test_engine.py</code>	6	Evidence Pack structure; reproducibility + ledger hash chain; <code>verify_run</code> reproducible; resolved execution parameters; threshold FAIL -> PASS; a missing source produces an ERROR record without crashing and verify handles it.

Run: `python tests/test_schema.py / test_connectors.py / test_engine.py`, or `python -m pytest tests/`. All 38 pass; `pyflakes` is clean.

4 Worked example — one run end to end

The seeded catalogue has seven rules (DQ01–DQ07) over the orders dataset, which contains six deliberate anomalies.

Rule	Dimension	What it checks	Result on the sample	Metric
DQ01	Completeness	orders.amount not null	FAIL — 1 null (ORD002)	completeness_pct 90.91
DQ02	Validity	status in {PENDING, SHIPPED, DELIVERED, CANCELLED}	FAIL — 'UNKNOWN_STATUS' (ORD006)	valid_pct 90.91
DQ03	Uniqueness	order_id unique	FAIL — ORD001 duplicated	duplicate_pct 18.18
DQ04	Consistency	customer_id exists in customers	FAIL — CUST999 orphan (ORD005)	orphan_pct 9.09
DQ05	Timeliness	order_date within 3 days of today	FAIL — ORD007 stale	within_sla_pct 81.82
DQ06	Reconciliation	orders.amount vs orders_reference.amount, 0.5% tol.	FAIL — ORD010 (999.99 vs 899.99)	reconciled_pct 90.91
DQ07	Validity	region in {FR, BE, DE}, threshold 2%	PASS — 0 invalid (within_threshold)	valid_pct 100.0

The run produces evidence/<run_id>/ with: run_summary.json (7 results, run_hash, previous_run_hash), seven DQ0x_result.json, six DQ0x_exceptions.csv (DQ07 has none), snapshots/orders.csv, snapshots/customers.csv, snapshots/orders_reference.csv, system_trace.json, run_log.jsonl + run.log. dqcompass report then writes the nine reporting artefacts (dataset_score for orders ~ 89–90, four High-severity entries in alerts.json). dqcompass verify <run_id> re-runs the seven controls against the snapshots and reports REPRODUCIBLE.

5 Brief-compliance matrix

Ref key: §n = use-case section n; A.x / B.x / C.x = Appendix A / B / C. Every row is satisfied by the prototype; caveats are stated inline and collected in section 6.

5.1 Core principle & mission (§1.2, §2.1, §2.3)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Controls defined, executed and evidenced in a structured manner	§1.2	Definition in a 21-column catalogue; execution by a single generic engine; evidence as a structured Evidence Pack per run.	control_catalogue.csv; dq_engine.py; evidence/<run_id>/	YES
Outputs traceable to input data and rule definitions	§1.2	Each result stores the dataset SHA-256 + a full copy, and the rule config snapshot; the run hash chains to the previous run.	<rule>_result.json, snapshots/ runs_ledger.jsonl	YES
Execution repeatable and reproducible	§1.2	Deterministic engine; two runs give identical results (tested); verify_run re-executes from snapshots and asserts equality.	tests/test_engine.py; verify_run(); verification.json	YES
Control failures captured, prioritised and actionable	§1.2	Exceptions written per rule; severity on every result; alerts.json lists failing High-severity controls with the catalogue's remediation action.	<rule>_exceptions.csv; reporting/alerts.json	YES
A generic control layer: applied across datasets, configurable with minimal effort, scalable across contexts	§2.1	One row adds a rule, one YAML entry adds a dataset, one class adds a connector — no engine code changes. Generic control functions take only column names.	controls.py; connectors.py; datasets_config.yaml	YES
Parameterised, reusable, centrally governed via a catalogue, producing documented evidence	§2.3	params JSON drives behaviour; the catalogue is the single governed repository; every run documents itself.	control_catalogue.csv; rule_authoring/; evidence/	YES

5.2 Solution design principles (§3)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Separation of concerns — rule definition separated from execution	§3.1	The catalogue + datasets_config define <i>what</i> and <i>where</i> ; dq_engine + controls run <i>how</i> . The engine never contains a rule or a path.	control_catalogue.csv; datasets_config.yaml; dq_engine.py	YES
Configurability — parameterised thresholds, fields, conditions; one rule to many datasets	§3.2	Every control is parameterised (field, keys, regex, min/max, max_lag_days, tolerance_pct, threshold_pct). check_completeness serves any field of any dataset.	params column; controls.py	YES
Standardisation — consistent control structure; standardised outputs	§3.3	Fixed 21-column schema; every control returns the same {status, metrics, exceptions} shape; every run writes the same Evidence Pack layout.	store.CANONICAL_COLUMNS; controls.py; evidence/<run_id>/	YES
Traceability — each execution provides source dataset, rule applied, timestamp, result	§3.4	Every <rule>_result.json carries dataset_reference (hash), control_function, rule_configuration_snapshot, execution_timestamp_utc, status + metrics.	<rule>_result.json	YES
Auditability — artefacts reviewable independently; allow reconstruction; demonstrate effectiveness	§3.5	The Evidence Pack is self-contained; verify_run reconstructs and re-checks; the run ledger and changelogs are hash-chained.	evidence/; verify_run(); *_changelog.jsonl	YES
Plug-and-play architecture — integrate into any EUC with minimal technical change	§3	Drop two files next to the EUC and call DQEngine(...).run(); or add dqcompass run to the pipeline.	docs/DEPLOYMENT.md; cli.py	YES

5.3 Functional architecture (§4)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Engine executes multiple control types	§4.1	Six control types dispatched from CONTROL_REGISTRY by control_type.	controls.py; dq_engine.py	YES
Engine applies rules across datasets dynamically	§4.1	Rules are data; the engine iterates the catalogue and resolves each dataset through a connector at run time.	dq_engine.run()	YES
Engine produces structured outputs; execution consistent and repeatable; results include severity + detailed metrics	§4.1	JSON results + CSV exceptions; deterministic; severity + a metrics dict on every result.	<rule>_result.json; tests/test_engine.py	YES
Control Catalogue: Rule ID, Description, Control Type, Logic definition, Severity, Owner+frequency, Expected output	§4.2	All present as named columns (see A.2 map in 5.7); logic_definition generated as pseudo-SQL.	control_catalogue.csv	YES
Catalogue: full traceability between rule definition and execution; reuse across datasets	§4.2	rule_id links definition <-> every result; dataset_scope is a parameter, not code.	<rule>_result.json; catalogue	YES
Reporting: DQ scorecards (traffic light)	§4.3	scorecard.csv/.html with GREEN/ORANGE/RED (+ GREY for ERROR); the app renders the same as a donut and per-rule badges.	reporting/scorecard.*	YES
Reporting: exception reports	§4.3	Per-rule <rule>_exceptions.csv + consolidated exceptions_detail.csv + exception_summary.csv (grouped).	reporting/exception*	YES
Reporting: control coverage views	§4.3	coverage.csv (datasets x dimensions) and the coverage heatmap on Home / Catalogue.	reporting/coverage.csv; app	YES
Reporting: clear identification of issues; drill-down to record level; consistent structure	§4.3	Traffic light + counts; exceptions_detail.csv is record-level; the schema of every report is stable across runs.	reporting/	YES
Audit Layer: execution logs; input dataset references; rule configuration snapshot; exception datasets	§4.4	run_log.json/run.log; dataset hash + snapshot; rule_configuration_snapshot; <rule>_exceptions.csv.	evidence/<run_id>/	YES
Audit Layer: evidence reproducible; outputs time-stamped and traceable; support independent audit validation	§4.4	verify_run() proves reproducibility; UTC timestamps everywhere; verification.json is the independent-validation artefact.	verify_run(); verification.json	YES

5.4 Deliverables (§5)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Working solution implementing DQ controls	§5.1	The engine runs the seven-rule catalogue over three datasets; 38 tests pass.	dq_engine.py; tests/	YES
Reusable control catalogue	§5.1	The 21-column CSV, authored and validated through the web app / dqcompass catalogue-validate.	control_catalogue.csv; rule_authoring/	YES
Automated control execution engine	§5.1	dqcompass run / cron / Airflow / CI; no manual step.	cli.py; docs/DEPLOYMENT.md	YES
Standard dashboards and reports; clear representation of DQ metrics	§5.2	The web dashboard + scorecard.html + the eight reporting CSV/JSON files (Power BI ready).	rule_authoring/; reporting/	YES
Documentation: architecture & design choices; control logic; limitations & scalability	§5.3	README.md, docs/ARCHITECTURE.md (4-layer model, data flow, limitations & scalability), this report, and the in-app Help page.	docs/ARCHITECTURE.md; this PDF	YES
Presentation: demo of the solution; business value & scalability	§5.4	The web app is the live demo (run, drill into evidence, verify, sign off); business value in §8.4 rows below.	rule_authoring/ (serve)	YES
Deliverables demonstrate the end-to-end lifecycle and clear auditable outputs	§5.5	Definition (Catalogue) -> Execution (Runs) -> Reporting (Runs/Mapping) -> Audit (Evidence Pack + verify) are all visible in one application.	the whole system	YES

5.5 Catalogue detail & expectations (§6, App. A.4)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Minimum fields: Rule ID, Control Type, Logic, Dataset Scope, Severity, Frequency, Owner, Output	§6	All present as dedicated columns; Logic held both as generated pseudo-SQL (logic_definition) and as machine params.	control_catalogue.csv	YES
Rules explicit, testable and reusable	§6 / A.4	Each rule is validated by ~30 checks before saving; column existence is probed against the real dataset; parameters make it reusable.	rule_authoring/schema.py	YES
All rules produce quantifiable outputs (KPIs)	§6 / A.4	kpi column per rule; every result carries a metrics dict with the quantitative measure.	kpi column; <rule>_result.json	YES
Controls support prioritisation / escalation through severity levels	§6 / A.4	severity on every rule and result; traffic light and alerts.json are severity-driven; dqcompass gate --severity escalates in CI.	severity; scorecard; alerts.json; gate	YES
Controls independent of datasets (reusable)	A.4	The six control functions receive only column names; the same function runs on any dataset.	controls.py	YES

5.6 Execution, evidence & audit expectations (§7)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Controls executed automatically	§7.1	dqcompass run is non-interactive; DEPLOYMENT.md gives cron / Airflow / CI; the catalogue frequency is the declarative schedule input.	cli.py; docs/DEPLOYMENT.md	YES
Execution consistent across runs	§7.1	Deterministic; identical per-rule status / metrics / exception counts across runs (tested).	tests/test_engine.py::test_reproducibility_and_ledger_chain	YES
Results stored and versioned	§7.1	Each run is an immutable folder; runs_ledger.jsonl versions the sequence with a hash chain.	evidence/<run_id>; runs_ledger.jsonl	YES
Each execution produces: Run ID + timestamp	§7.2	UUID run_id + UTC ISO timestamp in run_summary.json and every result.	run_summary.json	YES
Each execution produces: Dataset reference (version or snapshot)	§7.2	SHA-256 hash per dataset and a byte-exact copy under snapshots/.	dataset_reference; snapshots/	YES
Each execution produces: Rule configuration used	§7.2	rule_configuration_snapshot + execution_parameters (resolved) in every result.	<rule>_result.json	YES
Each execution produces: Summary results (pass/fail + metrics)	§7.2	status + metrics per rule; aggregated counts in run_summary.json.	run_summary.json	YES
Each execution produces: Detailed exception records	§7.2	<rule>_exceptions.csv per failing rule; consolidated in reporting.	<rule>_exceptions.csv	YES
Reconstruction of historical control runs	§7.3	Every run folder is self-contained (config + data snapshots + logs + trace); the Runs page lists and opens any past run.	evidence/; Runs page	YES
Independent verification of results	§7.3	dqcompass verify <run_id> re-executes from stored inputs and asserts equality; writes verification.json; non-zero exit on mismatch.	verify_run(); verification.json	YES
Traceability from data input to control output	§7.3	dataset snapshot (+hash) -> control_function + params -> status + metrics + exceptions, all in one folder, chained by run_hash.	the Evidence Pack	YES

5.7 Appendix A.2 — the 14 mandatory catalogue attributes

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Rule ID	A.2	rule_id — unique, format-checked, immutable after creation.	col 1	YES
Control Name	A.2	control_name — 3–80 chars.	col 2	YES
Control Type	A.2	control_type — one of the six dimensions.	col 3	YES
Description	A.2	description — business objective, >=10 chars.	col 4	YES
Logic Definition (SQL / pseudo-code / formula)	A.2	logic_definition — generated pseudo-SQL, e.g. <code>SELECT * FROM orders WHERE amount IS NULL ...</code>	col 5	YES
Dataset Scope	A.2	dataset_scope — validated against datasets_config.yaml.	col 6	YES
Data Element	A.2	data_element — the field(s) tested; generated from params.	col 7	YES
Threshold (acceptance criteria / tolerance)	A.2	threshold (plain text, generated) + threshold_pct (numeric, applied by the engine).	cols 8, 18	YES
Severity (High / Medium / Low)	A.2	severity — controlled vocabulary.	col 9	YES
Frequency	A.2	frequency — 7-value controlled vocabulary.	col 10	YES
Owner	A.2	owner — required.	col 11	YES
Output Type	A.2	output_type — suggestions provided; free value allowed.	col 12	YES
KPI (measurement indicator)	A.2	kpi — required; the metrics dict carries the value.	col 13	YES
Remediation Action	A.2	remediation_action — >=10 chars; carried into alerts.json.	col 14	YES

5.8 Appendix B.2 — the 10 mandatory Evidence components

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Run ID	B.2	UUID in run_summary.json and every result.	run_summary.json	YES
Execution Timestamp	B.2	UTC ISO-8601, run and per rule.	*_result.json	YES
Dataset Reference (version / snapshot / hash)	B.2	SHA-256 per dataset + exact copy under snapshots/.	snapshots/ dataset_reference	YES
Rule Configuration (full definition)	B.2	rule_configuration_snapshot per result.	*_result.json	YES
Execution Parameters (thresholds & config)	B.2	execution_parameters — resolved values incl. reference_date_resolved, threshold_pct.	*_result.json	YES
Control Results (summary metrics, pass/fail, KPI values)	B.2	status + metrics per rule; counts aggregated.	run_summary.json	YES
Exception Dataset (detailed failing records)	B.2	One CSV per failing rule.	*_exceptions.csv	YES
Logs (execution logs, processing steps)	B.2	run_log.jsonl (structured) + run.log (text): every step time-stamped.	run_log.jsonl; run.log	YES
System Trace (execution engine traceability)	B.2	system_trace.json: engine / Python / lib versions, host, file SHA-256s, control fn per rule.	system_trace.json	YES
Sign-off Fields (optional validation / certification)	B.2	signoff.json — reviewed_by / reviewed_at / decision / comment, recorded from the app.	signoff.json	YES

Appendix B.3 — evidence lifecycle

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
Generation — automatically produced at execution	B.3	The engine writes the whole pack in run(); no manual step.	dq_engine.run()	YES
Storage — structured, retrievable	B.3	One folder per run under evidence/; the Runs page and the ledger index them.	evidence/ runs_ledger.jsonl	YES
Traceability — linked to dataset, rule, execution context	B.3	dataset hash + snapshot, rule_id + config snapshot, run_id + run_hash + previous_run_hash.	the Evidence Pack	YES
Reproducibility — re-run controls with identical outputs	B.3	verify_run() re-executes from the stored config + snapshots and asserts equality.	verify_run(); verification.json	YES
Audit Review — independent reconstruction of the control	B.3 / B.4	Everything needed is in the folder; verification.json is the machine-checkable reconstruction.	evidence/<run_id>/	YES

5.9 Appendix C — supervisory mapping & principles

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
C.2 mapping: Requirement -> Control Implemented -> Evidence Generated -> Output	C.2	mapping.py emits one row per active rule with exactly these columns; requirement from regulatory_ref or a per-dimension default.	reporting/supervisory_mapping.csv; Mapping page	YES
C.3 control-to-evidence traceability (rule -> evidence -> KPI -> output)	C.3	Each mapping row also carries logic_definition, kpi, output and the last run id + status + run_hash for that rule.	supervisory_mapping.csv; Mapping page links to the run	YES
C.4 Traceability — control linked to rule definition, dataset, execution run	C.4	rule_id <-> result; dataset hash + snapshot; run_id + run_hash.	the Evidence Pack	YES
C.4 Repeatability — consistent results under identical inputs	C.4	Deterministic engine; verify_run proves it per run.	verify_run()	YES
C.4 Auditability — independent parties can reconstruct execution and validate outcomes	C.4	Self-contained run folders + verification.json + the Runs drill-down.	evidence/; Runs page	YES
C.4 Governance — controls owned, monitored, actionable	C.4	owner on every rule; coverage matrix + dashboard monitor; remediation_action + alerts.json; hash-chained changelogs.	catalogue; app; alerts.json; *_changelog.jsonl	YES

5.10 Success criteria (§8) & closing note (§9)

Brief requirement	Ref	How DQ Compass satisfies it	Evidence / artefact	Met
8.1 Technical quality — robustness & reliability of the engine	§8.1	38 automated tests incl. engine-level; the engine never crashes on bad input (ERROR result); atomic catalogue writes.	tests/; dq_engine.py	YES
8.1 Technical quality — scalability across datasets	§8.1	Adding a dataset / rule / connector is O(1) config, no engine change. The pandas execution can be swapped for a push-down backend behind the same function signatures (documented).	connectors.py; docs/ARCHITECTURE.md §5	YES
8.2 DQ maturity — coverage of the key DQ dimensions	§8.2	All six dimensions implemented; coverage matrix + coverage.csv show which datasets are covered on which dimension.	controls.py; reporting/coverage.csv	YES
8.2 DQ maturity — sophistication & reusability of controls	§8.2	Parameterised generic functions; tolerance thresholds; composite keys; referential integrity; reconciliation with tolerance; a severity-weighted dataset score.	controls.py; dataset_score.csv	YES
8.3 Governance & auditability — strength of the control catalogue	§8.3	21-column schema; ~30 pre-save validation checks; controlled vocabularies; column probing.	rule_authoring/schema.py	YES
8.3 Governance & auditability — quality of evidence generation	§8.3	All 10 B.2 components; hash-chained ledger; independent verification; sign-off.	evidence/; verify_run()	YES
8.3 Governance & auditability — traceability & transparency	§8.3	Two hash-chained changelogs; the Activity timeline; the Appendix C mapping links each control to its last run.	*_changelog.jsonl; Mapping page	YES
8.4 Business value — ability to reduce operational risk	§8.4	Fragmented manual checks become one governed, evidenced, auditable layer; failures are prioritised and routed with a remediation action.	the whole system; alerts.json	YES
8.4 Business value — ease of deployment across EUCs	§8.4	Two config files + one function call, or one CLI line; <code>pip install -e ..</code>	docs/DEPLOYMENT.md; pyproject.toml	YES
8.4 Business value — potential for industrialisation	§8.4	CLI, CI quality gate, scheduler adapters, notification recipe, WORM-storage retention note.	cli.py; docs/DEPLOYMENT.md	YES
§9 — scales across use cases; embeds control logic into the data lifecycle; audit-ready outputs aligned with supervisory expectations	§9	Config-only onboarding; a CI gate embeds the checks in the pipeline; the Evidence Pack + Appendix C mapping are the audit-ready, supervisor-aligned outputs.	docs/DEPLOYMENT.md; mapping.py; evidence/	YES

6 Known limitations & scope boundaries

- **Connectors** — only CSV and Excel execute. JSON, SQL database and SharePoint/Drive/URL are shown as selectors and saved with status: preview; the engine skips them. The abstraction is complete (each is one class away from working).
- **Execution** — rules run sequentially with pandas in memory. For large volumes the control functions would be re-implemented against a push-down backend (DuckDB / SQL) behind the same signatures; the catalogue, evidence format and reporting would not change.
- **Scheduler** — there is no built-in scheduler. dqcompass run is designed to be called by cron / Airflow / CI; the catalogue frequency column is the declarative input a scheduler would consume.
- **Evidence storage** — local files. Production would point evidence/ at versioned WORM storage; the intent is recorded in every system_trace.json.
- **Scoring** — the per-rule verdict is pass/fail; dataset_score.csv adds a severity-weighted composite, and trend.csv a per-run series, but there is no automated run-over-run alerting beyond the CI gate.
- **Authoring app** — single local user, cookie-based editor name; no authentication (out of scope for the prototype).

7 How to run

```
pip install -r requirements.txt # pandas, pyyaml, flask, openpyxl
pip install -e . # optional: the `dqcompass` command

python migrate_catalogue.py # once (also regenerates derived columns)
python migrate_sources.py # once

python run_authoring.py # web app -> http://127.0.0.1:5001
# or: dqcompass serve

dqcompass run # execute + write the Evidence Pack
dqcompass report # generate the Reporting Layer files
dqcompass mapping # write supervisory_mapping.csv
dqcompass verify <run_id> # prove a past run is reproducible
dqcompass catalogue-validate # validate every rule
dqcompass gate --severity High # CI quality gate

python tests/test_schema.py # 21 tests
python tests/test_connectors.py # 11 tests
python tests/test_engine.py # 6 tests
```

Generated by docs/build_report.py using ReportLab. Source of truth for every claim is the code in this repository; see docs/ARCHITECTURE.md and docs/DEPLOYMENT.md for the design rationale and operational recipes.